

Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions

O artigo *Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions*, escrito por Xinyi Hou, Yanjie Zhao, Shenao Wang e Haoyu Wang da Universidade de Ciência e Tecnologia de Huazhong, propõe uma análise sistemática do MCP sob duas perspectivas complementares: arquitetural e de segurança. O trabalho vai além de descrever o protocolo em si, mapeando seu ecossistema atual, os riscos que ele introduz e os desafios que precisam ser enfrentados para garantir sua adoção sustentável.

O MCP foi lançado pela Anthropic em 2024 como uma resposta à fragmentação existente na integração de ferramentas externas com modelos de linguagem. Antes de sua criação, cada integração exigia implementações específicas por plataforma, autenticações manuais e lógicas de execução próprias. O protocolo resolve isso ao oferecer uma camada padronizada de descoberta, invocação e comunicação bidirecional entre agentes de IA e serviços externos. A analogia com o *Language Server Protocol* (LSP) da área de IDEs é elucidativa: assim como o LSP desacoplou editores de linguagens de programação, o MCP desacopla modelos de suas ferramentas.

A estrutura arquitetural do MCP é organizada em três componentes principais *host*, *client* e *server* que colaboram para que o modelo identifique a intenção do usuário, selecione a ferramenta adequada e execute operações em sistemas externos. O *MCP server* expõe três tipos de capacidades: *tools* (ações em serviços externos), *resources* (acesso a dados) e *prompts* (templates reutilizáveis). Essa divisão lembra a separação de responsabilidades que estudamos em padrões de projeto, como a distinção entre camadas de serviço, repositório e apresentação.

Um dos pontos centrais do artigo é a definição do ciclo de vida de um servidor MCP em quatro fases: criação, implantação, operação e manutenção. Cada fase possui atividades específicas e, mais importante, superfícies de ataque próprias. Essa visão orientada ao ciclo de vida me pareceu especialmente útil, pois conecta problemas práticos de desenvolvimento como declarações de capacidade imprecisas ou instaladores não verificados a riscos concretos de segurança. Em projetos nos quais trabalhei, frequentemente nos preocupávamos apenas com a fase de operação, negligenciando vulnerabilidades introduzidas já na criação ou na implantação do sistema.

A taxonomia de ameaças apresentada é um dos pontos mais ricos do texto. Os autores categorizam 16 cenários de ataque distribuídos entre quatro tipos de agentes: desenvolvedores maliciosos, atacantes externos, usuários maliciosos e falhas de segurança estruturais. Entre os ataques mais preocupantes, destaco o *Tool Poisoning*, no qual instruções maliciosas são embutidas nos campos de descrição de uma ferramenta e interpretadas pelo modelo como diretivas legítimas. Esse tipo de ataque é particularmente insidioso porque ocorre após a seleção da ferramenta o sistema retorna resultados corretos ao usuário enquanto executa ações ocultas em segundo plano, como exfiltração de arquivos ou chamadas a endpoints externos. Em disciplinas de Arquitetura de Software,

discutimos bastante o princípio do menor privilégio, e esse cenário ilustra de forma concreta o que acontece quando ele é ignorado.

Outro ataque que considero especialmente relevante para o contexto atual é o *Indirect Prompt Injection*. Nele, o atacante insere instruções maliciosas em conteúdo externo como uma *issue* pública de um repositório GitHub que será recuperado pelo servidor MCP durante a execução de uma tarefa aparentemente inofensiva. O modelo processa essas instruções como parte do fluxo legítimo, sem distingui-las de dados confiáveis. Esse cenário mostra como a confiança depositada em fontes externas pode ser explorada de maneira quase invisível, e por que mecanismos de sanitização de saídas de ferramentas são tão necessários quanto a validação de entradas do usuário.

O artigo também analisa o ecossistema atual do MCP de forma empírica. Os autores identificaram mais de 26 coleções públicas de servidores, com dezenas de milhares de entradas. Ao amostrar aleatoriamente 300 servidores listados na plataforma MCP.so, encontraram que uma parcela significativa estava em desenvolvimento ativo, indisponível ou sequer relacionada ao protocolo. Isso evidencia um problema de maturidade: o crescimento da comunidade é acelerado, mas a qualidade e a verificação dos servidores são desiguais. Esse desequilíbrio entre adoção rápida e governança fraca lembra discussões que tivemos em sala sobre os riscos de usar dependências sem auditoria em projetos de software.

As recomendações propostas pelos autores são estruturadas fase a fase. Na criação, sugerem automação da geração de metadados e validação de declarações de capacidade antes da publicação. Na implantação, defendem o uso de ambientes *sandbox* com permissões mínimas e verificação criptográfica dos pacotes. Na operação, recomendam monitoramento em tempo real e gerenciamento de sessões com tokens de curta duração. Na manutenção, propõem controle de versão rigoroso e auditorias contínuas de logs. Essa estrutura progressiva reforça a ideia de que segurança não é uma etapa isolada, mas uma propriedade que deve ser incorporada em cada momento do desenvolvimento.

Do ponto de vista da formação em Engenharia de Software, o artigo amplia a percepção sobre o que significa projetar sistemas interoperáveis. O MCP representa uma mudança de paradigma relevante: deixamos de ter integrações ponto a ponto e passamos a um ecossistema de serviços compostáveis e desdobráveis. Porém, essa comparabilidade introduz superfícies de ataque que não existiam antes. Conceitos como encadeamento de ferramentas (*tool chaining*) mostram que uma sequência de operações individualmente legítimas pode, em conjunto, resultar em uma exfiltração completa de dados sensíveis o que nenhum controle de acesso isolado seria capaz de prevenir.

Model Context Protocol oferece uma contribuição valiosa tanto pela análise técnica do protocolo quanto pela visão crítica dos riscos que ele introduz. O texto é relevante para estudantes de Engenharia de Software porque conecta conceitos arquiteturais — ciclos de vida, separação de responsabilidades, princípio do menor privilégio — a um problema real e atual: a segurança de sistemas de IA que interagem com o mundo externo. Mais do que um catálogo de ameaças, o artigo fornece um framework de raciocínio sobre segurança que pode ser aplicado a qualquer protocolo de integração. Compreender esses mecanismos é

essencial para quem pretende projetar ou manter sistemas que incorporem agentes de IA de forma responsável.